

- 核心战斗玩法框架
 - 总述（面试用）
 - 五层框架
 - 第1层：输入与目标识别
 - 第2层：行为调度
 - 第3层：AI 决策系统
 - 第4层：战斗执行
 - 第5层：武器与数值
 - 关键文件速查
- 数据持久化系统
 - 总述（面试用）
 - 核心架构：三层分离
 - 机制一：GUID 唯一标识
 - 机制二：ISaveable 接口 + SaveableEntity 聚合
 - 机制三：BinaryFormatter 二进制序列化
 - 跨场景传送与续存
 - 关键文件速查
- 背包与交互系统
 - 总述（面试用）
 - 核心数据模型
 - 背包 (Inventory)：slot + number 结构体数组
 - 装备槽 (Equipment)：字典映射装备位 → 物品
 - 快捷键栏 (ActionStore)：字典映射 0-5 → ActionItem
 - 拖拽系统：IDragContainer 泛型框架
 - 三个接口
 - DragItem 的核心流程
 - 交换算法 (AttemptSwap)
 - 各槽位的 IDragContainer 实现
 - 事件驱动 UI 刷新
 - 世界交互：拾取与自动装备
 - 物品基类层级
 - 关键文件速查
- 任务与对话系统
 - 总述（面试用）
 - 整体架构
 - 机制一：任务数据模型
 - 机制二：条件系统 (IPredicateEvaluator + Condition)
 - 机制三：对话引擎 (PlayerConversant)
 - 机制四：对话 ↔ 任务桥接 (DialogueTrigger + Action 字符串)
- 可视化对话编辑器
 - 总述（面试用）
 - 核心架构：三层分离
 - 机制一：ScriptableObject 子资产嵌套存储
 - 为什么用 GUID 字符串而非直接引用？
 - 子资产生命周期
 - 删除时的清理

- 机制二：贝塞尔曲线连线
- 机制三：Undo/Redo 全操作覆盖
- 机制四：事件驱动的事件交互
 - 坐标系统
 - 两种拖拽，同一套事件处理
 - 延迟操作模式
- 机制五：link/unlink 四态切换
- 对话节点类型
- 关键文件速查
- 网络通信模块
 - 总述（面试用）
 - 整体架构
 - 机制一：二进制协议格式 —— 4 字节 ID + 4 字节 Length + Body
 - 协议定义（XML 驱动）
 - 序列化（Writing）—— 以 LoginMsg 为例
 - 反序列化（Reading）—— 对称读取
 - 机制二：分包/粘包处理
 - 机制三：线程模型 —— 子线程收包入队，主线程出队分发
 - 机制四：协议 ID 路由 —— MsgPool + 反射
 - 机制五：LoginManager 事件解耦网络层与 UI
 - 机制六：心跳保活 + DontDestroyOnLoad
 - 代码生成流程
 - 关键文件速查

核心战斗玩法框架

总述（面试用）

核心战斗玩法基于 **NavMesh 寻路** 实现点击移动与 AI 导航，通过 **IAction 行为调度** 保证同一时刻只有一个行为在执行。敌人 AI 在 **巡逻 / 追击 / 警戒** 三种状态间切换，**AggroGroup 群体仇恨联动** 实现「一敌发现，全员响应」。战斗系统沿「**索敌 → 追击 → 攻击动画 → 伤害结算 → 死亡**」这一完整循环展开，武器与数值通过 **ScriptableObject + IModifierProvider** 实现数据驱动和模块化扩展。

五层框架

第1层	输入与目标识别	← 鼠标点了什么？地面还是敌人？
第2层	行为调度	← 移动和攻击不能同时发生
第3层	AI 决策系统	← 敌人：巡逻 → 发现 → 追击 → 丢失 → 警戒 → 回位
第4层	战斗执行	← 索敌→追击→攻击动画→伤害结算→死亡
第5层	武器与数值	← 数据在哪？数值怎么算？

第1层：输入与目标识别

鼠标射线穿透场景，IRaycastable 接口多态分发 —— 同一个点击动作，根据命中对象的不同执行不同的逻辑。

讲解：

玩家每帧的操作入口是 `PlayerController.Update()`。它按优先级依次尝试：

1. UI 拦截? → 点击在 UI 上，不处理 3D 世界
2. 命中组件? → 遍历 `IRaycastable` 接口，由命中对象自己决定怎么响应
3. 命中地面? → 调用 `Mover` 移动

核心机制是 `InteractWithComponent()` —— 它对鼠标射线做 `RaycastAll`（穿透所有碰撞体），按距离排序后遍历每个命中物体上的所有 `IRaycastable` 组件。**谁先处理谁胜出**（return true 就停止）。

`IRaycastable` 接口只有两个方法：

- `HandleRaycast(PlayerController)` —— 执行点击逻辑（攻击、拾取、对话...）
- `GetCursorType()` —— 告诉 `PlayerController` 当前该显示什么光标

`CombatTarget` 实现了 `IRaycastable`：被点击时调用 `Fighter.Attack()`，光标切为战斗图标。拾取物、NPC 对话也是同样的模式 —— **这是典型的策略模式：PlayerController 不关心你点了什么，由被点击的对象自己决定。**

```
// PlayerController.cs - 组件交互的优先级分发
private bool InteractWithComponent()
{
    RaycastHit[] hits = RaycastAllSorted(); // 穿透排序
    foreach (RaycastHit hit in hits)
    {
        IRaycastable[] raycastables = hit.transform.GetComponents<IRaycastable>();
        foreach (IRaycastable raycastable in raycastables)
        {
            if (raycastable.HandleRaycast(this)) // 谁先处理谁赢
            {
                SetCursorType(raycastable.GetCursorType());
                return true;
            }
        }
    }
    return false;
}
```

```
// CombatTarget.cs - 敌人被点击时
public bool HandleRaycast(PlayerController callingController)
{
    if (!callingController.GetComponent<Fighter>().CanAttack(gameObject))
        return false;
}
```

```
if (Input.GetMouseButtonDown(0))
    callingController.GetComponent<Fighter>().Attack(gameObject);

return true;
}
```

移动点击则是 `InteractWithMovement()`：射线先打到物理碰撞，再用 `NavMesh.SamplePosition` 投影到 NavMesh 上，确保目标点在可行走区域内。`Mover.CanMoveTo()` 还会校验路径是否完整、长度是否超过最大距离（40f），太远的点击会被拒绝。

第2层：行为调度

IAction 接口 + ActionScheduler —— 同一时刻只有一个“主动行为”在运行，启动新行为自动取消旧行为。

讲解：

这是整个系统的「中央调度器」。`IAction` 是一个极简接口：

```
public interface IAction
{
    void Cancel();
}
```

`Mover` 和 `Fighter` 都实现了 `IAction`。`ActionScheduler` 组件挂载在每个角色上，维护一个 `currentAction` 引用：

```
// ActionScheduler.cs
IAction currentAction;

public void StartAction(IAction action)
{
    if (currentAction == action) return; // 同一个 action，不重复启动
    if (currentAction != null)
        currentAction.Cancel();         // 取消旧的
    currentAction = action;              // 切换到新的
}
```

典型流程 —— 玩家在攻击时点击地面：

1. `Mover.StartMoveAction()` 调用 `ActionScheduler.StartAction(this)`
2. `ActionScheduler` 发现当前 action 是 `Fighter`，调用 `Fighter.Cancel()`
3. `Fighter.Cancel()` → 清空攻击目标、触发 `stopAttack` 动画、取消移动
4. 新的 action 变成 `Mover`，开始寻路

为什么不用状态机？ 因为这里的状态只有两种（移动/战斗），IAction 模式比状态机更轻量，且天然可扩展——以后加「对话」「技能」只需新增一个实现了 IAction 的组件。

```
// Fighter.Cancel() — 取消战斗
public void Cancel()
{
    StopAttack();                // 动画触发器 "stopAttack"
    target = null;               // 清空目标
    GetComponent<Mover>().Cancel(); // 停止 NavMeshAgent
}
```

```
// Mover.Cancel() — 取消移动
public void Cancel()
{
    navMeshAgent.isStopped = true;
}
```

死亡时也是同一机制：Health.Die() 调用 ActionScheduler.CancelCurrentAction()，无论当时在做什么，全部停掉。

第3层：AI 决策系统

巡逻 → 追击 → 警戒，三态隐式状态机；SphereCast 群体喊话，一敌发现全员联动。

讲解：

AIController.Update() 每帧执行一个 if-else 链，本质上是一个隐式状态机，三个状态按优先级排列：

优先级	状态	条件	行为
最高	追击 (Chase)	玩家在 chaseDistance 内 或 被 Aggreivate 触发	调用 fighter.Attack(player) + 喊话附近敌人
中	警戒 (Suspicion)	刚丢失玩家视野，时间 < suspicionTime(3s)	停在原地，取消所有行为
最低	巡逻 (Patrol)	默认状态	沿 PatrolPath 走航点，或回到出生位置

```
// AIController.Update() — 隐式状态机核心
void Update()
{
    if (health.IsDead()) return;

    if (InAggreivated() && fighter.CanAttack(player))
        AttackBehaviour(); // 追击
    else if (timeSinceLastSawPlayer < suspicionTime)
```

```
SuspicionBehaviour(); // 警戒
else
    PatrolBehaviour(); // 巡逻

timeSinceLastSawPlayer += Time.deltaTime;
timeSinceAggrevated += Time.deltaTime;
}
```

群体仇恨联动 (AggroGroup):

`AIController.AttackBehaviour()` 中会调用 `AggrevateNearbyEnemies()`:

```
private void AggrevateNearbyEnemies()
{
    RaycastHit[] hits = Physics.SphereCastAll(
        transform.position, shoutDistance, Vector3.up, 0);
    foreach (RaycastHit hit in hits)
    {
        AIController ai = hit.collider.GetComponent<AIController>();
        if (ai == null) continue;
        ai.Aggrevate(); // 重置其 aggro 计时器
    }
}
```

`InAggrevated()` 的判定是双重条件:

```
private bool InAggrevated()
{
    return distanceToPlayer < chaseDistance
        || timeSinceAggrevated < aggravationCooldownTime; // 5秒冷却
}
```

这意味着: 一个敌人发现玩家 → SphereCast 喊话 → 半径内所有敌人的 `timeSinceAggrevated` 被设为 0 → 它们全部进入追击状态并持续至少 5 秒。形成「拉一群怪」的玩法体验。

另外还有 `AggroGroup` 组件 (`Combat/AggroGroup.cs`), 用于批量激活/停用一组敌人的 `Fighter` + `CombatTarget` 组件, 配合触发器使用 (玩家进入某区域后激活埋伏的敌人)。

第4层: 战斗执行

Fighter.Update 驱动「索敌 → 追击 → 攻击动画 → Hit() 伤害结算 → 死亡」完整循环; 远程/近战的分支发生在 `Hit()` 内部, 由 `WeaponConfig` 的 `projectile` 字段数据驱动。

讲解:

这是整个战斗系统的核心循环, 由 `Fighter.Update()` 每帧驱动:

```
// Fighter.Update() — 战斗循环的心脏
void Update()
{
    timeSinceLastAttack += Time.deltaTime;

    if (target == null) return;
    if (target.IsDead()) return;

    if (!GetIsInRange(target.transform))
        GetComponent<Mover>().MoveTo(target.transform.position, 1f); // 追击
    else
    {
        GetComponent<Mover>().Cancel(); // 进入射程，停下
        AttackBehavior(); // 攻击
    }
}
```

流程图：

```
索敌 (Attack) → 追击 (MoveTo) → 进射程 → 面向目标 → CD到了?
→ 触发攻击动画 (SetTrigger "attack")
→ 动画事件回调 Hit()
    → 获取伤害值 (BaseStats.GetStat)
    → 远程? → LaunchProjectile → 弹道飞行 → 碰撞触发 TakeDamage
    → 近战? → 直接 TakeDamage
→ 目标死亡? → Die() → 动画 + 取消行为 + 奖励经验
→ 回到索敌
```

攻击节奏由 **timeSinceLastAttack** 计时器 + **Animator** 动画事件控制，不是 Update 里每帧打伤害。攻击动画播放到特定帧时，动画事件回调 **Fighter.Hit()**：

```
// Fighter.Hit() — 动画事件回调，真正造成伤害的时机
void Hit()
{
    if (target == null) return;

    float damage = GetComponent<BaseStats>().GetStat(Stat.Damage);

    if (currentWeapon.value != null)
        currentWeapon.value.OnHit(); // 武器特效（音效、粒子）

    if (currentWeaponConfig.HasProjectile()) // 远程：发射弹道
        currentWeaponConfig.LaunchProjectile(rightHandTransform, target,
        gameObject, damage);
    else // 近战：直接结算
        target.TakeDamage(gameObject, damage);
}
```

近战 vs 远程的分叉：是在 `Hit()` 内部通过 `WeaponConfig.HasProjectile()` 判断的。远程武器配置了 `projectilePrefab`，走 `LaunchProjectile` 实例化弹道；近战的这个字段为空，直接 `TakeDamage`。**这是数据驱动，不是两套代码**——同一套 `Fighter` 同时支持近战和远程。

弹道 (Projectile)：实例化后朝目标飞行，`OnTriggerEnter` 检测到目标 `Health` 组件才结算伤害（不会误伤其他单位）。支持 `isHoming` 追踪模式，有 `maxLifeTime` 超时自毁。

死亡 (Health.Die)：

```
private void Die()
{
    isDead = true;
    onDie.Invoke(); // UnityEvent, 可供掉落等系统
    挂钩
    GetComponent<Animator>().SetTrigger("die"); // 播放死亡动画
    GetComponent<ActionScheduler>().CancelCurrentAction(); // 停止一切行为
}
```

死亡后：`Mover.Update()` 禁用 `NavMeshAgent`，`AIController.Update()` 第一行就 `return`，`Fighter` 看到 `target.IsDead()` 也停止操作。

第5层：武器与数值

ScriptableObject 定义武器模板，Progression 定义等级成长，IModifierProvider 让任意组件参与属性计算——最终汇聚为一个公式：(基础值 + 加算) × (1 + 乘算%)。

讲解：

武器系统 (WeaponConfig ScriptableObject)：

武器不是一个挂满 `MonoBehaviour` 的 `Prefab`，而是一个纯数据的 `ScriptableObject`：

```
[CreateAssetMenu(fileName = "Weapon", menuName = "Weapons/New Weapon")]
public class WeaponConfig : ScriptableObject
{
    AnimatorOverrideController animatorOverride; // 武器专属动画
    Weapon equippedPrefab; // 手上显示的模型
    float weaponRange = 2f; // 攻击距离
    float weaponDamage = 5f; // 基础伤害
    float percentageBonus = 0f; // 百分比加成
    bool isRightHanded = true; // 左手还是右手
    Projectile projectilePrefab = null; // 不为空 = 远程武器
}
```

`Spawn()` 是工厂方法：销毁旧武器模型 → 在手上实例化新模型 → 替换 `AnimatorOverrideController`。**换刀、换弓只是换一个 `ScriptableObject` 引用。**

武器的切换通过装备系统驱动：`Equipment.equipmentUpdated` 事件 → `Fighter.UpdateWeapon()` → 从装备槽取出 `WeaponItem` → 获取其 `WeaponConfig` → `EquipWeapon()`。

数值系统 (Progression + BaseStats + IModifierProvider):

`Progression` 是一张 `ScriptableObject` 数据表，按 职业 × 属性 × 等级 三维索引：

```
Progression
├─ CharacterClass.Player
│   ├─ Stat.Health:      [100, 120, 145, 175, ...] (每级)
│   ├─ Stat.Damage:      [8, 10, 13, 17, ...]
│   └─ Stat.Experience:  [0, 100, 250, 500, ...] (升到下一级所需)
├─ CharacterClass.Grund
└─ ...
```

`BaseStats.GetStat()` 是伤害公式的最终汇聚点：

```
public float GetStat(Stat stat)
{
    return (progression.GetStat(stat, characterClass, currentLevel)
        + GetAdditiveModifier(stat)) // 累加所有组件的加算修正
        * (1 + GetPercentageModifier(stat) / 100); // 累加所有组件的乘算修正
}
```

IModifierProvider 接口 —— 让任何组件都能参与属性修正：

```
public interface IModifierProvider
{
    IEnumerable<float> GetAdditiveModifier(Stat stat);
    IEnumerable<float> GetPercentageModifier(Stat stat);
}
```

`BaseStats` 通过 `GetComponents<IModifierProvider>()` 收集 `GameObject` 上所有实现了该接口的组件，遍历累加。**Fighter 就是其中一个 Provider**：它把当前武器的 `weaponDamage` 作为 `Stat.Damage` 的加算修正、`percentageBonus` 作为乘算修正。以后加 Buff/Debuff 系统，只需新增一个实现了 `IModifierProvider` 的组件挂在角色上即可，完全不改现有代码。

经验与等级闭环：

```
杀敌 → Health.AwardExperience() → Experience.GainExperience()
      → BaseStats.UpdateLevel() → 等级提升 → onLevelUp 事件
      → Health.RegenerateHealth() (回血到 70%) + LevelUpEffect()
```

关键文件速查

文件	角色
Control/PlayerController.cs	鼠标输入、射线检测、IRaycastable 分发
Control/IRaycastable.cs	点击多态接口
Control/AIController.cs	敌人三态 AI + 群体仇恨联动
Control/PatrolPath.cs	巡逻路径定义
Core/IAction.cs	可取消行为接口
Core/ActionScheduler.cs	行为调度器，保证互斥
Movement/Mover.cs	NavMeshAgent 封装，寻路与移动
Combat/Fighter.cs	战斗核心循环、武器切换、IModifierProvider
Combat/CombatTarget.cs	敌人可被点击攻击
Combat/WeaponConfig.cs	ScriptableObject 武器数据
Combat/Weapon.cs	武器模型 + OnHit 特效事件
Combat/Projectile.cs	远程弹道（追踪/直线）
Combat/AggroGroup.cs	批量激活/停用敌人组
Attributes/Health.cs	生命值、受伤、死亡、经验奖励
Stats/BaseStats.cs	属性计算中枢（公式汇聚点）
Stats/Progression.cs	ScriptableObject 职业×等级数值表
Stats/ImodifierProvider.cs	属性修正接口
Stats/Experience.cs	经验值追踪

数据持久化系统

总述（面试用）

通用存档系统基于 **GUID 唯一标识 + ISaveable 接口 + BinaryFormatter 二进制序列化** 三层设计：每个可存档对象持有一个编辑器自动分配的 GUID，SaveableEntity 聚合自身所有 ISaveable 组件，以「类型名 → 状态数据」的二级字典方式捕获状态；SavingSystem 统一调用 `FindObjectsOfType<SaveableEntity>()` 遍历全场景，将整个字典树通过 BinaryFormatter 序列化为 .sav 文件。存档同时记录 `lastSceneBuildIndex`，加载时先异步切场景再恢复状态，实现跨场景传送与续存。

核心架构：三层分离



设计要点: **SavingSystem** 不知道任何具体业务逻辑。它只认识 **SaveableEntity** 和 **ISaveable** 两个抽象, 不关心存的是血量还是装备。每个模块自行实现 **CaptureState()** / **RestoreState()**, 职责完全分离。

机制一: GUID 唯一标识

编辑器自动分配 GUID, 全局字典去重 —— 场景中有 100 个敌人, 存档里靠 GUID 找到属于「这一个」敌人的数据。

```

// SaveableEntity.cs
[ExecuteAlways]
public class SaveableEntity : MonoBehaviour
{
    [SerializeField] string uniqueIdentifier = "";
    static Dictionary<string, SaveableEntity> globalLookup;

    #if UNITY_EDITOR
    private void Update() {
        if (Application.isPlaying(gameObject)) return;
        // 编辑器下自动生成 GUID, 并确保全局唯一
        if (string.IsNullOrEmpty(property.stringValue) ||
            !IsUnique(property.stringValue))
            property.stringValue = System.Guid.NewGuid().ToString();
        globalLookup[property.stringValue] = this;
    }
    #endif
}
  
```

- 在编辑器中, 每个挂载了 **SaveableEntity** 的 GameObject 自动获得一个唯一的 GUID 字符串
- **static globalLookup** 确保全局不重复
- 运行时 GUID 已烘焙在场景/Prefab 中, 不再变化
- 加载时, SavingSystem 用 GUID 作为字典 key 匹配每个实体: "这个敌人在存档里的数据对应场景里的哪个 **GameObject**"

机制二: ISaveable 接口 + SaveableEntity 聚合

SaveableEntity 不存自己的数据，它遍历 **GameObject** 上所有 **ISaveable** 组件，以完整类型名为 key 构建二级字典。

```
// ISaveable.cs — 极简契约
public interface ISaveable
{
    object CaptureState();
    void RestoreState(object state);
}
```

```
// SaveableEntity.CaptureState() — 聚合所有组件
public object CaptureState()
{
    Dictionary<string, object> state = new Dictionary<string, object>();
    foreach (ISaveable saveable in GetComponents<ISaveable>())
    {
        state[saveable.GetType().ToString()] = saveable.CaptureState();
    }
    return state;
}

// SaveableEntity.RestoreState() — 按类型名分发
public void RestoreState(object state)
{
    Dictionary<string, object> stateDict = (Dictionary<string, object>)state;
    foreach (ISaveable saveable in GetComponents<ISaveable>())
    {
        string typeString = saveable.GetType().ToString();
        if (stateDict.ContainsKey(typeString))
            saveable.RestoreState(stateDict[typeString]);
    }
}
```

序列化后的数据结构：

```
{
  "a1b2c3d4-...": {                                ← GUID (SaveableEntity 的
uniqueIdentifier)
    "RPG.Attributes.Health": 45.0,                ← 类型名 → 组件状态
    "RPG.Movement.Mover":   {x:10, y:0, z:20},
    "RPG.Combat.Fighter":   "Sword",
  },
  "e5f6g7h8-...": {
    "RPG.Attributes.Health": 0.0,                ← 已经死了
    "RPG.Combat.Fighter":   "Axe",
  },
}
```

```
"lastSceneBuildIndex": 3
}
```

← 顶层特殊 key

各组件存的都是什么：

组件	CaptureState 返回	说明
Health	float (当前血量)	恢复时若为 0 则调用 Die()
Experience	float (经验值)	直接恢复
Mover	SerializableVector3 (位置)	先关 NavMeshAgent → 设位置 → 重开
Fighter	string (武器配置名)	通过 Resources.Load<WeaponConfig> 恢复
Inventory	SlotRecord[] (物品ID + 数量)	按 ID 查找物品填充背包
Equipment	Dictionary<EquipLocation, string>	按 ID 恢复装备槽
ActionStore	Dictionary<int, DockedItemRecord>	恢复快捷键栏 1-6
QuestList	List<object> (每个 QuestStatus)	恢复任务名 + 已完成目标
Shop	List<StockRecord> (已售出数量)	防止重复购买限量物品
PickupSpawner	bool (是否已被拾取)	已拾取则销毁
ItemDropper	DropRecord[] (掉落物位置+ID)	在保存位置重新生成掉落物

机制三： BinaryFormatter 二进制序列化

整个字典树一次序列化为 .sav 二进制文件，存到 Application.persistentDataPath。

```
// SavingSystem.cs — 核心三步骤

// 1. 捕获：遍历场景中所有 SaveableEntity
private void CaptureState(Dictionary<string, object> state)
{
    foreach (SaveableEntity saveable in FindObjectsOfType<SaveableEntity>())
        state[saveable.GetUniqueIdentifier()] = saveable.CaptureState();

    state["lastSceneBuildIndex"] = SceneManager.GetActiveScene().buildIndex;
}

// 2. 序列化：写入文件
private void SaveFile(string saveFile, object state)
{
    string path = Path.Combine(Application.persistentDataPath, saveFile + ".sav");
    using (FileStream stream = File.Open(path, FileMode.Create))
    {
```

```

        BinaryFormatter formatter = new BinaryFormatter();
        formatter.Serialize(stream, state);
    }
}

// 3. 恢复：从文件反序列化 → 按 GUID 分发
private void RestoreState(Dictionary<string, object> state)
{
    foreach (SaveableEntity saveable in FindObjectsOfType<SaveableEntity>())
    {
        string id = saveable.GetUniqueIdentifier();
        if (state.ContainsKey(id))
            saveable.RestoreState(state[id]);
    }
}

```

有容错处理：文件不存在、为空、或反序列化抛出异常（文件损坏）时，返回空字典，游戏以干净状态启动。

Save 的细节值得一提：`Save()` 不是直接覆盖，而是先 `LoadFile()` 读出旧存档 → `CaptureState()` 覆盖/补充当前场景的实体 → 再写回。这样跨场景的数据不会丢失 —— 从 Scene01 传送到 Scene02，Scene02 的存档操作不会抹掉 Scene01 中已保存的实体数据。

跨场景传送与续存

存档中记录 `lastSceneBuildIndex`，启动时异步加载对应场景，场景就绪后再恢复状态。

```

// SavingSystem.LoadLastScene()
public IEnumerator LoadLastScene(string saveFile)
{
    Dictionary<string, object> state = LoadFile(saveFile);
    int buildIndex = SceneManager.GetActiveScene().buildIndex;
    if (state.ContainsKey("lastSceneBuildIndex"))
        buildIndex = (int)state["lastSceneBuildIndex"];

    yield return SceneManager.LoadSceneAsync(buildIndex); // 先切场景
    RestoreState(state); // 再恢复
}

```

`SavingWrapper.Awake()` 中启动协程，配合 `Fader` 做淡入淡出：

```

IEnumerator LoadLastScene()
{
    yield return GetComponent<SavingSystem>().LoadLastScene(defaultSaveFile);
    Fader fader = FindObjectOfType<Fader>();
    fader.FadeOutImmediate(); // 黑屏遮罩
    yield return fader.FadeIn(0.2f); // 淡入
}

```

完整流程： 启动游戏 → 读取存档 → 获取 `lastSceneBuildIndex` → 异步加载该场景 → 黑屏 → 场景所有 `SaveableEntity` 就绪 → `RestoreState` 恢复位置、血量、装备、任务... → 淡入显示 → 玩家在存档时的场景和位置继续游戏。

关键文件速查

文件	角色
<code>Saving/SavingSystem.cs</code>	中央引擎：文件 I/O、全场景遍历、序列化/反序列化
<code>Saving/ISaveable.cs</code>	双方法接口契约
<code>Saving/SaveableEntity.cs</code>	GUID 生成与去重、ISaveable 组件聚合
<code>Saving/SerializableVector3.cs</code>	Vector3 的 BinaryFormatter 可序列化包装
<code>SceneManagement/SavingWrapper.cs</code>	高层 API、S/L/Del 快捷键、场景加载 + 淡入淡出
<code>SceneManagement/Fader.cs</code>	CanvasGroup 淡入淡出控制

背包与交互系统

总述（面试用）

背包系统以 `slot + number 结构体数组` 为数据核心，通过 `InventoryItem.IsStackable()` 控制同类堆叠（可堆叠物品叠加 number，不可堆叠物品独占一格）。UI 层采用 `IDragContainer` 泛型拖拽框架：从格子拖出即为 `IDragSource`，拖入格子即为 `IDragDestination`，一个格子同时实现两者即可支持 **交换（Swap）** ——先清空双方再互相填入，容量不够则回滚。装备系统维护 `Dictionary<EquipLocation, EquipableItem>`，独立的 `equipmentUpdated` 事件同时驱动装备 UI 刷新和 Fighter 武器切换。所有模型层变更均通过 **C# event 驱动 UI 全量重绘**（销毁重建），数据流单向透明：`模型变更 → 事件触发 → UI 响应`。

核心数据模型

背包（Inventory）：`slot + number 结构体数组`

存储 = 固定长度的 `InventorySlot[]`，每个 slot 是一个 `{item, number}` 结构体。找空位逻辑：先找同类堆叠 → 再找空格。

```
// Inventory.cs — 数据结构
public struct InventorySlot
{
    public InventoryItem item;
    public int number;
}
InventorySlot[] slots; // 默认 32 格

// 找空位：优先堆叠，其次空格
```

```
private int FindSlot(InventoryItem item)
{
    int i = FindStack(item);    // 同类 + IsStackable → 叠加 number
    if (i < 0)
        i = FindEmptySlot();    // 第一个 item==null 的格子
    return i;                    // -1 表示背包满
}

private int FindStack(InventoryItem item)
{
    if (!item.IsStackable()) return -1;    // 不可堆叠, 跳过
    for (int i = 0; i < slots.Length; i++)
        if (object.ReferenceEquals(slots[i].item, item))
            return i;
    return -1;
}
```

关键设计点:

- 堆叠判断用 `object.ReferenceEquals` 而不是 ID 比较 —— 同一个 ScriptableObject 实例才算同一物品
- `InventoryItem.IsStackable()` 是 ScriptableObject 上的 bool 字段, 策划在资源上勾选
- `AddToFirstEmptySlot` 和 `AddItemToSlot` 每次成功后都 `inventoryUpdated?.Invoke()`

装备槽 (Equipment): 字典映射装备位 → 物品

```
// Equipment.cs
Dictionary<EquipLocation, EquipableItem> equippedItems;

public void AddItem(EquipLocation slot, EquipableItem item)
{
    Debug.Assert(item.GetAllowedEquipLocation() == slot);    // 类型检查
    equippedItems[slot] = item;
    equipmentUpdated?.Invoke();
}
```

`EquipLocation` 枚举: `Helmet, Necklace, Body, Trousers, Boots, Weapon, Shield, Gloves`。

快捷键栏 (ActionStore): 字典映射 0-5 → ActionItem

```
// ActionStore.cs
Dictionary<int, DockedItemSlot> dockedItems;    // key: 0~5

public bool Use(int index, GameObject user)
{
    dockedItems[index].item.Use(user);            // 执行效果
    if (dockedItems[index].item.isConsumable())
        RemoveItems(index, 1);                    // 消耗品用一次少一个
}
```



```
    return true;
}
```

PlayerController 每帧检测按键 1-6 → `actionStore.Use(0..5, gameObject)`。

拖拽系统：IDragContainer 泛型框架

DragItem<T> 处理所有指针事件，通过 **IDragSource<T>** / **IDragDestination<T>** 接口与具体槽位解耦。一个格子同时实现两个接口 = **IDragContainer**，天然支持交换。

三个接口

```
IDragSource<T>           // 拖出: GetItem(), GetNumber(), RemoveItems(n)
IDragDestination<T>      // 拖入: MaxAcceptable(item), AddItems(item, n)
IDragContainer<T> : IDragDestination<T>, IDragSource<T> // 双向 = 可交换
```

DragItem 的核心流程

```
// 拖拽开始 → 脱离原父节点，挂到根 Canvas 上，禁用射线遮挡
void OnBeginDrag() {
    transform.SetParent(parentCanvas.transform);           // 自由移动
    GetComponent<CanvasGroup>().blocksRaycasts = false;    // 让射线穿透到下方的槽位
}

// 拖拽结束 → 回原位，恢复射线，查找目标容器
void OnEndDrag() {
    transform.SetParent(originalParent);                   // 归位
    GetComponent<CanvasGroup>().blocksRaycasts = true;

    IDragDestination<T> container;
    if (!EventSystem.current.IsPointerOverGameObject())
        container = parentCanvas.GetComponent<IDragDestination<T>>(); // 丢到世界
    else
        container =
eventData.pointerEnter.GetComponentInParent<IDragDestination<T>>(); // 丢到槽位

    if (container != null) DropItemIntoContainer(container);
}
```

交换算法（AttemptSwap）

1. 暂存：记录 source 和 destination 的 {item, number}
2. 清空：双方都 RemoveItems
3. 计算回退量：如果对方 MaxAcceptable 放不下全部，算出要退回多少

4. 容量校验：如果最终交换违反任一方限制 → 全部回滚
5. 正式交换：互相 AddItems

```
private void AttemptSwap(IDragContainer<T> destination, IDragContainer<T> source)
{
    // 1. 暂存 & 清空
    var srcItem = source.GetItem(); var srcNum = source.GetNumber();
    var dstItem = destination.GetItem(); var dstNum = destination.GetNumber();
    source.RemoveItems(srcNum);
    destination.RemoveItems(dstNum);

    // 2. 计算对方放不下的部分，退回原主
    var srcTakeBack = CalculateTakeBack(srcItem, srcNum, source, destination);
    // ...

    // 3. 容量不够 → 全部回滚
    if (source.MaxAcceptable(dstItem) < dstNum ||
        destination.MaxAcceptable(srcItem) < srcNum)
    {
        destination.AddItems(dstItem, dstNum); // 回滚
        source.AddItems(srcItem, srcNum);
        return;
    }

    // 4. 正式交换
    source.AddItems(dstItem, dstNum);
    destination.AddItems(srcItem, srcNum);
}
```

各槽位的 IDragContainer 实现

槽位类	实现接口	MaxAcceptable 逻辑
InventorySlotUI	IDragContainer<InventoryItem>, IItemHolder	背包有空位 → int.MaxValue，否则 0
EquipmentSlotUI	IDragContainer<InventoryItem>, IItemHolder	匹配 EquipLocation 且槽为空 → 1，否则 0
ActionSlotUI	IDragContainer<InventoryItem>, IItemHolder	非 ActionItem → 0；同物品且消耗品 → MaxValue；同物品非消耗品 → 0；空槽 → 1
InventoryDropTarget	IDragDestination<InventoryItem>	永远 int.MaxValue（丢出背包 = 掉落到世界）

三者都是「薄适配层」：不存数据，所有操作直接代理给 Model（Inventory / Equipment / ActionStore）。

事件驱动 UI 刷新

模型层只发事件，不关心 UI 如何响应。UI 层订阅事件，全量销毁重建 —— 简单粗暴但 foolproof。

```
Inventory.AddToFirstEmptySlot() → inventoryUpdated
Inventory.RemoveFromSlot() → inventoryUpdated
↓ 唯一的订阅者
InventoryUI.Redraw()
→ 销毁所有子物体
→ 遍历 32 格，每格 Instantiate(InventorySlotUI) + Setup(inventory, index)
```

```
// InventoryUI.cs — 事件驱动的全量刷新
private void Awake()
{
    playerInventory.inventoryUpdated += Redraw;
}

private void Redraw()
{
    foreach (Transform child in transform)
        Destroy(child.gameObject); // 全部销毁

    for (int i = 0; i < playerInventory.GetSize(); i++)
    {
        var itemUI = Instantiate(InventoryItemPrefab, transform);
        itemUI.Setup(playerInventory, i); // 重建
    }
}
```

三个事件的订阅链：

```
inventoryUpdated (Inventory)
    ↳ InventoryUI.Redraw()

equipmentUpdated (Equipment)
    ↳ EquipmentSlotUI.RedrawUI()
    ↳ Fighter.UpdateWeapon() ← 切换武器时自动重装备

storeUpdated (ActionStore)
    ↳ ActionSlotUI.UpdateIcon()
```

设计思路：为什么不增量更新（只改变动的格子）？因为全量销毁重建在这个规模（32 格）下性能完全够用，而增量更新需要维护「哪个格子变了」的状态同步逻辑，容易出 bug。**先用最简单方案，性能出问题再优化。**

世界交互：拾取与自动装备

点击世界中的拾取物 → Pickup.PickupItem() → 优先尝试自动装备 → 否则放入背包 → 销毁 GameObject。

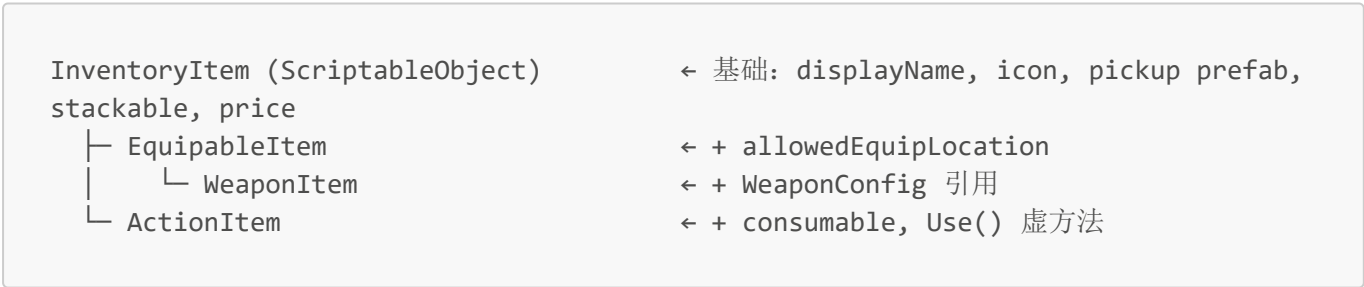
```
// Pickup.cs
public void PickupItem()
{
    if (TryAutoEquip()) // 是可装备物 + 对应槽位为空?
    {
        Destroy(gameObject); // 直接装上, 不占背包
        return;
    }

    bool foundSlot = inventory.AddToFirstEmptySlot(item, number);
    if (foundSlot)
        Destroy(gameObject); // 放入背包
}

private bool TryAutoEquip()
{
    var equipable = item as EquipableItem;
    if (equipable == null) return false;
    // 对应装备槽为空 → 自动装备
    return equipment.GetItemInSlot(equipable.GetAllowedEquipLocation()) == null;
}
```

ClickablePickup 实现 IRaycastable, 被点击时调用 pickup.PickupItem()。这复用了第 1 层战斗框架的 InteractWithComponent() 射线分发机制。

物品基类层级



InventoryItem.GetFromID(string itemID) 是静态工厂方法：首次调用时 Resources.LoadAll<InventoryItem>("") 全量加载并缓存为字典，后续按 ID 查找。存档恢复、拖拽交换都靠这个方法把字符串 ID 转回物品引用。

关键文件速查

文件	角色
Inventories/InventoryItem.cs	物品基类 ScriptableObject (ID、名称、图标、堆叠、价格)
Inventories/EquipableItem.cs	可装备物品 (+ 装备槽位限制)

文件	角色
Inventories/WeaponItem.cs	武器物品（包装 WeaponConfig）
Inventories/ActionItem.cs	技能/消耗品（+ consumable + Use()）
Inventories/Inventory.cs	背包数据：slot+number 数组 + inventoryUpdated 事件
Inventories/Equipment.cs	装备数据：EquipLocation 字典 + equipmentUpdated 事件
Inventories/ActionStore.cs	快捷键栏：0-5 字典 + storeUpdated 事件
Inventories/Pickup.cs	世界拾取物：自动装备 / 放入背包
Inventories/PickupSpawner.cs	场景中拾取物生成器（ISaveable）
Inventories/ItemDropper.cs	掉落物品到世界
Control/ClickablePickup.cs	IRaycastable → 点击拾取
Utils/UI/Dragging/IDragSource.cs	拖出接口
Utils/UI/Dragging/IDragDestination.cs	拖入接口
Utils/UI/Dragging/IDragContainer.cs	双向接口（支持交换）
Utils/UI/Dragging/DragItem.cs	拖拽行为（BeginDrag/Drag/EndDrag + Swap 算法）
UI/Inventories/InventorySlotUI.cs	背包格子：IDragContainer + IItemHolder
UI/Inventories/InventoryUI.cs	背包面板：订阅 inventoryUpdated → 全量重建
UI/Inventories/EquipmentSlotUI.cs	装备格子：IDragContainer + IItemHolder
UI/Inventories/ActionSlotUI.cs	快捷键格子：IDragContainer + IItemHolder
UI/Inventories/InventoryDropTarget.cs	Canvas 级拖放目标：拖到空白处 = 丢到世界
UI/Inventories/InventoryItemIcon.cs	物品图标 + 堆叠数量显示
UI/Inventories/IItemHolder.cs	接口：提供 GetItem() 给 Tooltip
UI/Inventories/ItemTooltipSpawner.cs	鼠标悬停显示物品 Tooltip

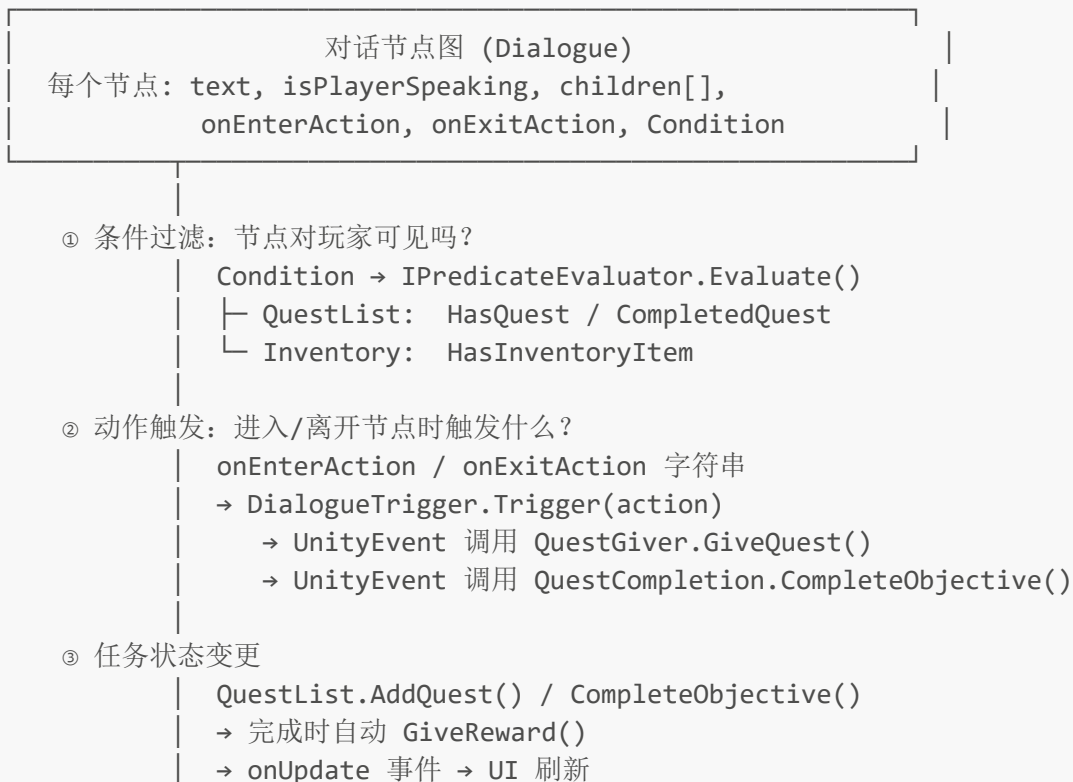
任务与对话系统

总述（面试用）

任务系统以 **Quest ScriptableObject**（目标列表 + 奖励列表）为数据定义，**QuestStatus**（持有 Quest 引用 + 已完成目标列表）为运行时状态，**QuestList** 挂载在玩家上统一管理。对话系统基于 **节点图（Dialogue Node Graph）**：每个节点携带文本、玩家/AI 发言标记、子节点引用、**onEnterAction/onExitAction** 动作字符串，以及一个 **Condition**（谓词 + 参数）。**IPredicateEvaluator** 接口连接条件系统与游戏状态：**Condition.Check()** 遍历玩家身上所有 **IPredicateEvaluator** 组件，调用 **Evaluate(predicate, parameters)** —— 返回 **null** 表示「不归

我管」、`false` 表示「不满足」、`true` 表示「满足」。对话节点通过 `onEnterAction/onExitAction` 字符串触发 **DialogueTrigger**，后者用 UnityEvent 桥接到 **QuestGiver**（接任务）和 **QuestCompletion**（完成目标），实现「对话驱动任务流转」。

整体架构



机制一：任务数据模型

Quest = **ScriptableObject** 静态定义（目标 + 奖励），**QuestStatus** = 运行时对象（持有 **Quest** + 已完成目标列表），**QuestList** = 玩家身上的管理器（**List** + **IPredicateEvaluator**）。

```
// Quest.cs — ScriptableObject, 策划在编辑器里配置
public class Quest : ScriptableObject
{
    List<Objective> objectives; // 每个: {string reference, string description}
    List<Reward> rewards;      // 每个: {int number, InventoryItem item}

    public static Quest GetByName(string questName) {
        foreach (Quest quest in Resources.LoadAll<Quest>(""))
            if (quest.name == questName) return quest;
        return null;
    }
}

// QuestStatus.cs — 普通 C# 类（不是 MonoBehaviour），运行时追踪单个任务进度
public class QuestStatus
```

```

{
    Quest quest;
    List<string> completedObjectives; // 已完成的 reference 列表

    public bool IsComplete() {
        foreach (var obj in quest.GetObjectives())
            if (!completedObjectives.Contains(obj.reference))
                return false;
        return true; // 所有目标都完成
    }

    public void CompleteObjective(string objective) {
        if (quest.HasObjective(objective))
            completedObjectives.Add(objective);
    }
}

// QuestList.cs — 挂在 Player 上
public class QuestList : MonoBehaviour, ISaveable, IPredicateEvaluator
{
    List<QuestStatus> statuses = new List<QuestStatus>();
    public event Action onUpdate;

    public void AddQuest(Quest quest) {
        if (HasQuest(quest)) return;
        statuses.Add(new QuestStatus(quest));
        onUpdate?.Invoke();
    }

    public void CompleteObjective(Quest quest, string objective) {
        QuestStatus status = GetQuestStatus(quest);
        status.CompleteObjective(objective);
        if (status.IsComplete())
            GiveReward(quest); // 所有目标完成, 自动发奖
        onUpdate?.Invoke();
    }

    private void GiveReward(Quest quest) {
        foreach (var reward in quest.GetRewards()) {
            bool success = GetComponent<Inventory>
            ().AddToFirstEmptySlot(reward.item, reward.number);
            if (!success)
                GetComponent<ItemDropper>().DropItem(reward.item, reward.number);
        }
    }
}

```

数据流: Quest (ScriptableObject 资源) → QuestStatus (new 出来的运行时对象) → QuestList.statuses (List 管理) → 存档序列化为 QuestStatusRecord[]

机制二：条件系统 (IPredicateEvaluator + Condition)

Condition 是「谓词 + 参数」的序列化结构，**Check()** 遍历所有 **IPredicateEvaluator**，返回 **null** 表示不相关（跳过）、**false** 表示不满足（短路）、全部通过则返回 **true**。这是一种责任链 + 三值逻辑的复合模式。

```
// Condition.cs — 可序列化，嵌入在 DialogueNode 中
[System.Serializable]
public class Condition
{
    string predicate;          // 如 "HasQuest", "CompletedQuest", "HasInventoryItem"
    string[] parameters;      // 如 ["RatQuest"], ["Sword_01"]

    public bool Check(IEnumerable<IPredicateEvaluator> evaluators) {
        foreach (var evaluator in evaluators) {
            bool? result = evaluator.Evaluate(predicate, parameters);
            if (result == null) continue;    // 不归我管，下一个
            if (result == false) return false; // 短路：有一个说不满足就不通过
        }
        return true; // 全部通过（或无人认识此谓词）
    }
}
```

```
// IPredicateEvaluator.cs — 三值逻辑接口
public interface IPredicateEvaluator
{
    bool? Evaluate(string predicate, string[] parameters);
    // null = "我不认识这个谓词"（责任链继续）
    // true = "条件满足"
    // false = "条件不满足"（立即短路）
}
```

两个实现者：

```
// QuestList 处理任务相关谓词
public bool? Evaluate(string predicate, string[] parameters) {
    switch (predicate) {
        case "HasQuest":
            return HasQuest(Quest.GetByName(parameters[0]));
        case "CompletedQuest":
            return GetQuestStatus(Quest.GetByName(parameters[0])).IsComplete();
    }
    return null; // 不归我管
}

// Inventory 处理物品相关谓词
public bool? Evaluate(string predicate, string[] parameters) {
    switch (predicate) {
```



```

        case "HasInventoryItem":
            return HasItem(InventoryItem.GetFromID(parameters[0]));
    }
    return null;
}

```

三值逻辑的设计理由： 如果只用 bool (true/false)，每个评估器必须对所有谓词表态。加了 null (不相关) 后，新增评估器只关心自己懂的谓词，不需要知道其他系统的存在。**符合开闭原则。**

在对话中的应用（条件性对话选项）：

- 玩家接了「杀鼠任务」→ QuestList 的 `HasQuest("RatQuest")` 返回 true → 节点 "老鼠杀完了吗？" 的条件通过 → 该选项显示
- 玩家没接任务 → 同样条件返回 false → 该选项隐藏
- 玩家接了且已完成 → `CompletedQuest("RatQuest")` 返回 true → 「交任务」选项显示

机制三：对话引擎 (PlayerConversant)

PlayerConversant 是对话的核心状态机： 根节点 → `Next()` 推进 → 有玩家选项则进入选择态 → `SelectChoice()` 进入下一节点 → 直到无子节点结束。每步推进都触发 `onEnterAction/onExitAction` + 条件过滤。

```

// PlayerConversant.cs — 核心对话流程
public void StartDialogue(AIConversant newConversant, Dialogue dialogue) {
    currentConversant = newConversant;
    currentDialogue = dialogue;
    currentNode = currentDialogue.GetRootNode();
    TriggerEnterAction(); // 进入根节点时触发动作
    onConversationUpdated?.Invoke();
}

public void Next() {
    // 1. 有玩家选项 → 进入选择态
    int numPlayerResponses = FilterOnCondition(
        currentDialogue.GetPlayerChildren(currentNode)).Count();
    if (numPlayerResponses > 0) {
        isChoosing = true;
        TriggerExitAction(); // 离开当前节点
        onConversationUpdated?.Invoke();
        return;
    }

    // 2. 随机选一个 AI 子节点推进
    DialogueNode[] children = FilterOnCondition(
        currentDialogue.GetAIChildren(currentNode)).ToArray();
    if (children.Length == 0) {
        currentNode = null; // 对话结束
        isChoosing = false;
        return;
    }
}

```

```

        TriggerExitAction();
        currentNode = children[Random.Range(0, children.Length)];
        TriggerEnterAction(); // 进入新节点
        onConversationUpdated?.Invoke();
    }

    public void SelectChoice(DialogueNode chosenNode) {
        currentNode = chosenNode;
        TriggerEnterAction();
        isChoosing = false;
        Next(); // 选完后继续推进
    }

    // 条件过滤 — 核心：不满足条件的节点直接不可见
    private IEnumerable<DialogueNode> FilterOnCondition(IEnumerable<DialogueNode>
inputNode) {
        foreach (var node in inputNode)
            if (node.CheckCondition(GetEvaluators())) // ← 这里调用 Condition.Check()
                yield return node;
    }

    private IEnumerable<IPredicateEvaluator> GetEvaluators() {
        return GetComponents<IPredicateEvaluator>(); // 从 Player 身上收集所有评估器
    }

```

机制四：对话 ↔ 任务桥接（DialogueTrigger + Action 字符串）

这是整个对话驱动游戏流程的关键桥接。**DialogueNode** 携带 **onEnterAction** / **onExitAction** 字符串，**PlayerConversant** 在触发时调用当前 NPC 身上所有 **DialogueTrigger** 组件，匹配 action 字符串后执行 **UnityEvent**。**UnityEvent** 在 **Inspector** 里绑定到 **QuestGiver.GiveQuest()** 或 **QuestCompletion.CompleteObjective()**。

```

// DialogueTrigger.cs — 将 action 字符串映射到 UnityEvent
public class DialogueTrigger : MonoBehaviour
{
    [System.Serializable]
    public class ActionEvent {
        public string action; // 如 "give_quest", "complete_kill_rats"
        public UnityEvent onTrigger; // Inspector 里绑到 QuestGiver /
QuestCompletion
    }

    [SerializeField] List<ActionEvent> actions;

    public void Trigger(string actionToTrigger) {
        foreach (var a in actions)
            if (a.action == actionToTrigger)
                a.onTrigger.Invoke();
    }
}

```

```
// PlayerConversant.TriggerAction() — 桥接的两端
private void TriggerAction(string action) {
    if (action == "") return;
    foreach (DialogueTrigger trigger in
currentConversant.GetComponents<DialogueTrigger>())
        trigger.Trigger(action); // 同一个 NPC 上可能有多个 DialogueTrigger
}

// QuestGiver.cs — 这个方法的调用由 UnityEvent 在 Inspector 中绑定
public void GiveQuest() {
    QuestList questList =
GameObject.FindGameObjectWithTag("Player").GetComponent<QuestList>();
    questList.AddQuest(quest);
}

// QuestCompletion.cs — 同理
public void CompleteObjective() {
    QuestList questList =
GameObject.FindGameObjectWithTag("Player").GetComponent<QuestList>();
    questList.CompleteObjective(quest, objective);
}
```

完整链路示例 —— 玩家接「杀老鼠」任务：

1. 玩家点击 NPC → AIConversant.HandleRaycast()
 - PlayerConversant.StartDialogue(this, dialogue)
 - 进入根节点（NPC 说"你好，冒险者"）
2. 玩家点 Next → FilterOnCondition 计算子节点
 - QuestList.Evaluate("HasQuest", ["RatQuest"]) → false（还没接）
 - "关于老鼠..." 选项的条件中不依赖 HasQuest，可见
 - "我来交任务" 选项的条件 [CompletedQuest: RatQuest] → false，隐藏
3. 玩家选"关于老鼠..." → SelectChoice → 进入该节点
 - onEnterAction = "give_rat_quest"
 - TriggerAction("give_rat_quest")
 - NPC 上的 DialogueTrigger.Trigger("give_rat_quest")
 - UnityEvent → QuestGiver.GiveQuest()
 - QuestList.AddQuest(ratQuest) ← 任务接到
4. 对话继续...节点 [HasQuest: RatQuest] 的条件现在为 true
 - 后续对话选项「我搞定老鼠了」变为可见

可视化对话编辑器

总述（面试用）

对话节点以 **ScriptableObject** 子资产形式内嵌存储，节点间通过 **GUID** 字符串引用关联；采用 **Handles.DrawBezier** 绘制贝塞尔曲线连线，支持节点拖拽、画布平移及完整的 **Undo/Redo** 操作。

核心架构：三层分离

```

DialogueEditor (EditorWindow)    ← 视图层: IMGUI 绘制 + Event 交互
    |
    ▼ 操作 (CreateNode / DeleteNode / SetPosition)
Dialogue (ScriptableObject)      ← 容器层: 节点集合 + GUID→节点字典查询
    |
    ▼ 包含 (CreateInstance + AddObjectToAsset)
DialogueNode (ScriptableObject)  ← 数据层: text、rect、children[],
isPlayerSpeaking
  
```

设计要点：**DialogueEditor** 不直接操作节点数据。所有修改都通过 **Dialogue** 的方法（**CreateNode**、**DeleteNode**）或 **DialogueNode** 的方法（**SetPosition**、**AddChild**、**RemoveChild**）间接完成，数据层方法内部统一处理 Undo 记录和脏标记，视图层只管绘制和事件分发。

机制一：ScriptableObject 子资产嵌套存储

每个 **DialogueNode** 通过 **CreateInstance<DialogueNode>()** 动态创建，以 **AssetDatabase.AddObjectToAsset(node, this)** 嵌入 **Dialogue** 资产文件。节点之间不直接引用对象，而是通过 **name**（**GUID** 字符串）建立父子关系。

为什么用 GUID 字符串而非直接引用？

直接引用（**List<DialogueNode> children**）有两个致命问题：删除节点后其他节点持有 dangling reference；Unity 序列化不支持 **ScriptableObject** 跨资产循环引用。用 GUID 字符串做间接引用，**Dialogue** 维护一个 **Dictionary<string, DialogueNode> nodeLookup** 做实时查询：

```

// Dialogue.cs — GUID → 节点查询
Dictionary<string, DialogueNode> nodeLookup = new Dictionary<string, DialogueNode>
();

private void OnValidate()
{
    nodeLookup.Clear();
    foreach (DialogueNode node in GetAllNodes())
        nodeLookup[node.name] = node; // name 即 GUID
}

public IEnumerable<DialogueNode> GetAllChildren(DialogueNode parentNode)
{
    foreach (string childID in parentNode.GetChildren())
        if (nodeLookup.ContainsKey(childID))
  
```

```

        yield return nodeLookup[childID]; // 按 GUID 反查节点对象
    }

```

子资产生命周期

```

// Dialogue.cs — 创建节点（工厂方法）
private DialogueNode MakeNode(DialogueNode parent)
{
    DialogueNode newNode = CreateInstance<DialogueNode>(); // ① 运行时创建，尚未关联资产文件
    newNode.name = Guid.NewGuid().ToString(); // ② 分配全局唯一标识

    if (parent != null)
    {
        parent.AddChild(newNode.name); // ③ 父节点记录 GUID
        newNode.SetPlayerSpeaking(!parent.IsPlayerSpeaking());
        newNode.SetPosition(parent.GetRect().position + newNode.Offset); // ④ 偏移 250px
    }
    return newNode;
}

// Dialogue.cs — 节点嵌入资产文件
public void OnBeforeSerialize()
{
    #if UNITY_EDITOR
        if (AssetDatabase.GetAssetPath(this) != "")
        {
            foreach (DialogueNode node in GetAllNodes())
            {
                if (AssetDatabase.GetAssetPath(node) == "")
                    AssetDatabase.AddObjectToAsset(node, this); // ⑤ 子资产嵌入
            }
        }
    #endif
}

```

关键时间点：`ISerializationCallbackReceiver.OnBeforeSerialize()` 在以下时机触发：

- 按 `Ctrl+S` 保存项目
- `AssetDatabase.SaveAssets()`
- Unity 编辑器失去焦点时

这意味着：在编辑器中通过 `+` 按钮创建的新节点，一开始只是内存中的 `CreateInstance` 对象，等到下次序列化时才自动挂到 Dialogue 资产下。如果中途关闭编辑器未保存，子节点不会变成孤儿文件污染 `AssetDatabase`。

删除时的清理

```

public void DeleteNode(DialogueNode nodeToDelete)
{
    Undo.RecordObject(this, "Deleted Dialogue Node");
    nodes.Remove(nodeToDelete);           // ① 从列表中移除
    OnValidate();                         // ② 重建查找字典
    CleanDanglingChildren(nodeToDelete);   // ③ 遍历所有节点，清除对该 GUID
的引用
    Undo.DestroyObjectImmediate(nodeToDelete); // ④ 销毁子资产对象
}

private void CleanDanglingChildren(DialogueNode nodeToDelete)
{
    foreach (DialogueNode node in GetAllNodes())
        node.RemoveChild(nodeToDelete.name); // 遍历清理 dangling 引用
}

```

机制二：贝塞尔曲线连线

用 `Handles.DrawBezier()` 绘制节点间的有向连线，控制点偏移算法保证曲线始终水平弯曲。

```

// DialogueEditor.cs — 连线绘制
private void DrawConnections(DialogueNode node)
{
    Vector3 startPosition = new Vector2(node.GetRect().xMax,
node.GetRect().center.y); // 右中点

    foreach (DialogueNode childNode in selectedDialogue.GetAllChildren(node))
    {
        Vector3 endPosition = new Vector2(childNode.GetRect().xMin,
childNode.GetRect().center.y); // 左中点

        // 控制点：只在 X 轴偏移，Y 保持水平 → 产生水平的 S 形曲线
        Vector3 controlPointOffset = endPosition - startPosition;
        controlPointOffset.y = 0;           // Y 归零，保证水平
        controlPointOffset.x *= 0.8f;       // 偏移量为距离的 80%

        Handles.DrawBezier(
            startPosition,                    // 起点（父节点右边缘）
            endPosition,                      // 终点（子节点左边缘）
            startPosition + controlPointOffset, // 控制点1
            endPosition - controlPointOffset,   // 控制点2
            Color.white,                      // 白色线条
            null,                             // 无纹理
            4f);                             // 线宽 4px
    }
}

```

绘制顺序是关键：OnGUI 中先遍历画所有连线，再遍历画所有节点。IMGUI 是画家算法——后画的在上层，所以节点会自然遮挡连线端点，视觉效果干净。

机制三：Undo/Redo 全操作覆盖

每次数据修改操作被 `Undo.RecordObject()` 包裹，配合 `Undo.RegisterCreatedObjectUndo()` 和 `Undo.DestroyObjectImmediate()` 覆盖创建/删除场景，形成完整的撤销闭环。

整个编辑器中涉及三类 Undo 操作：

操作	Undo API	撤销效果
拖拽节点	<code>Undo.RecordObject(selectedDialogue, "Move Dialogue Node")</code>	恢复节点位置
修改文本	<code>Undo.RecordObject(this, "Update Dialogue Text")</code>	恢复文本内容
link/unlink	<code>Undo.RecordObject(this, "Add/Remove Dialogue Link")</code>	恢复父子关系
创建节点	<code>Undo.RegisterCreatedObjectUndo(newNode, ...) +</code> <code>Undo.RecordObject(this, ...)</code>	撤销创建 → 自动销毁子资产
删除节点	<code>Undo.RecordObject(this, ...) +</code> <code>Undo.DestroyObjectImmediate(nodeToDelete)</code>	撤销删除 → 还原原子资产

为什么 `CreateNode` 和 `DeleteNode` 需要两步 Undo？因为操作涉及两个对象：

- `Dialogue`（父资产）：`nodes` 列表变了，需要 `RecordObject`
- `DialogueNode`（子资产）：对象本身创建/销毁了，需要 `RegisterCreatedObjectUndo / DestroyObjectImmediate`

```
// Dialogue.cs — 创建节点时的双重 Undo
public void CreateNode(DialogueNode parent)
{
    DialogueNode newNode = MakeNode(parent);
    Undo.RegisterCreatedObjectUndo(newNode, "Created Dialogue Node"); // 记录子资产的创建
    Undo.RecordObject(this, "Added Dialogue Node"); // 记录父资产的列表变更
    AddNode(newNode);
}

// Dialogue.cs — 删除节点时的双重 Undo
public void DeleteNode(DialogueNode nodeToDelete)
{
    Undo.RecordObject(this, "Deleted Dialogue Node"); // 先记录父资产
    nodes.Remove(nodeToDelete);
    OnValidate();
    CleanDanglingChildren(nodeToDelete);
    Undo.DestroyObjectImmediate(nodeToDelete); // 再销毁子资产
}
```

机制四：事件驱动的节日交互

Event.current 提供每帧的输入事件，通过「鼠标位置 → 画布坐标转换 → Rect.Contains 命中检测」实现节点拖拽和画布平移。延迟操作模式避免在绘制循环中修改集合。

坐标系统

`EditorGUILayout.BeginScrollView(scrollPosition)` 改变了绘制原点。鼠标位置是窗口坐标，节点位置是画布坐标，转换公式：**画布坐标 = 窗口坐标 + scrollPosition**。

```
// 命中检测 — 窗口坐标转画布坐标
private DialogueNode GetNodeAtPoint(Vector2 point)
{
    foreach (DialogueNode node in selectedDialogue.GetAllNodes())
        if (node.GetRect().Contains(point)) // point 已是画布坐标
            return node;
    return null;
}

// 调用处:
draggingNode = GetNodeAtPoint(Event.current.mousePosition + scrollPosition);
```

两种拖拽，同一套事件处理

```
private void ProcessEvents()
{
    // 按下：判断点在节点上还是空白区域
    if (Event.current.type == EventType.MouseDown && draggingNode == null)
    {
        draggingNode = GetNodeAtPoint(Event.current.mousePosition + scrollPosition);
        if (draggingNode != null)
        {
            draggingOffset = draggingNode.GetRect().position - Event.current.mousePosition;
            Selection.activeObject = draggingNode; // Inspector 显示节点属性
        }
        else
        {
            draggingCanvas = true;
            draggingCanvasOffset = Event.current.mousePosition + scrollPosition;
            Selection.activeObject = selectedDialogue; // Inspector 显示 Dialogue
        }
    }
    // 拖拽节点 → 更新 Rect.position
    else if (Event.current.type == EventType.MouseDrag && draggingNode != null)
    {
        Undo.RecordObject(selectedDialogue, "Move Dialogue Node");
        draggingNode.SetPosition(Event.current.mousePosition + draggingOffset);
    }
}
```

资产


```

        GUI.changed = true;
    }
    // 拖动画布 → 反向更新 scrollPosition
    else if (Event.current.type == EventType.MouseDrag && draggingCanvas)
    {
        scrollPosition = draggingCanvasOffset - Event.current.mousePosition;
        GUI.changed = true;
    }
    // 松开 → 重置状态
    else if (Event.current.type == EventType.MouseUp && draggingNode != null)
        draggingNode = null;
    else if (Event.current.type == EventType.MouseUp && draggingCanvas)
        draggingCanvas = false;
}

```

延迟操作模式

在 OnGUI 中遍历节点绘制时，不能直接调用 `CreateNode` / `DeleteNode` 修改 `nodes` 列表（会导致 `Collection was modified` 异常）。解决方案：用标志变量延迟到绘制循环结束后执行。

```

// 绘制循环中只设置标志
if (GUILayout.Button("+")) { creatingNode = node; }
if (GUILayout.Button("x")) { deletingNode = node; }

// OnGUI 末尾统一执行
if (creatingNode != null) { selectedDialogue.CreateNode(creatingNode);
creatingNode = null; }
if (deletingNode != null) { selectedDialogue.DeleteNode(deletingNode);
deletingNode = null; }

```

机制五：link/unlink 四态切换

非模态链接操作：不在弹出窗口中操作，直接在节点面板上用按钮完成「选择父节点 → 标记子节点」的连线流程。

`DrawLinkButtons()` 实现了一个简洁的四态状态机：

```

状态1 [初始] 一点 link → 状态2 [已选父节点]
状态2        一点 cancel → 回到状态1（点击同一节点）
状态2        一点 unlink → 状态3 [解除已有链接] → 回到状态1
状态2        一点 child  → 状态4 [创建新链接]    → 回到状态1

```

```

private void DrawLinkButtons(DialogueNode node)
{
    if (linkingParentNode == null) // 状态1：初始
    {

```

```
        if (GUILayout.Button("link"))
            linkingParentNode = node;
    }
    else if (linkingParentNode == node) // 状态2: 取消选择
    {
        if (GUILayout.Button("cancel"))
            linkingParentNode = null;
    }
    else if (linkingParentNode.GetChildren().Contains(node.name)) // 状态3: 已是子
节点
    {
        if (GUILayout.Button("unlink"))
        {
            linkingParentNode.RemoveChild(node.name);
            linkingParentNode = null;
        }
    }
    else // 状态4: 确认链接
    {
        if (GUILayout.Button("child"))
        {
            linkingParentNode.AddChild(node.name);
            linkingParentNode = null;
        }
    }
}
```

对话节点类型

- **AI 节点** (`isPlayerSpeaking = false`): NPC 说的话, 父节点有多个 AI 子节点时随机选一个
- **玩家节点** (`isPlayerSpeaking = true`): 玩家的回复选项, 多个玩家子节点时进入选择态 (`isChoosing = true`)

关键文件速查

文件	角色
Dialogue/Editor/DialogueEditor.cs	可视化编辑器窗口: ImGui 绘制、事件处理、Undo 集成
Dialogue/Dialogue.cs	对话节点图容器: GUID → 节点查询、CreateNode/DeleteNode、子资产管理
Dialogue/DialogueNode.cs	单个对话节点数据: text、rect、children[]、isPlayerSpeaking、Condition
Dialogue/PlayerConversant.cs	对话引擎 (推进、选择、条件过滤、动作触发)
Dialogue/AIConversant.cs	NPC 上的对话入口 (IRaycastable → StartDialogue)
Dialogue/DialodueTrigger.cs	Action 字符串 → UnityEvent 桥接器

文件	角色
Quest/Quest.cs	任务 ScriptableObject 定义（目标、奖励、GetByName）
Quest/QuestStatus.cs	单个任务运行时状态（Quest 引用 + 已完成目标列表）
Quest/QuestList.cs	玩家任务管理器（List + IPredicateEvaluator）
Quest/QuestGiver.cs	给予任务（由 UnityEvent 调用）
Quest/QuestCompletion.cs	完成目标（由 UnityEvent 调用）
Core/IPredicateEvaluator.cs	三值逻辑条件接口（bool? Evaluate）
Core/Condition.cs	可序列化条件（predicate + parameters + Check）

网络通信模块

总述（面试用）

网络层基于 **SocketAsyncEventArgs 异步 TCP 长连接**，设计「**4 字节协议 ID + 4 字节 Body 长度 + Body**」的二进制协议头解决分包/粘包——缓存区累积接收数据后循环解析头部、校验 Body 完整性、完整则反射创建消息对象入队、不完整则挪到缓存区头部等待下次回调拼接。采用「**子线程收包入队、主线程 Update 出队分发**」的线程模型规避 Unity 主线程限制：接收回调在 .NET 线程池执行解析并入队 `Queue<BaseHandler>`，`Update()` 在主线程 `Dequeue().MsgHandle()` 安全调用 Unity API。协议 ID 通过 **MsgPool 字典** → **Activator.CreateInstance 反射** 路由至对应 Handler，新增协议只需在 XML 中定义字段 → 代码生成器生成 Message/Handler 类 → MsgPool 注册一行，NetAsyncMgr 不用改。通过 **LoginManager 事件机制**（`onLoginResult` / `onRegisterResult`）实现网络层与 UI 完全解耦：Handler 只调 `RaiseXxx()`，UI 只订阅事件。连接通过 `DontDestroyOnLoad` 跨场景存活，配合心跳每 2 秒保活。

整体架构



```

路由层 (MsgPool + BaseHandler)
msgPool.GetMessage(id) → Activator.CreateInstance
msgPool.GetHandler(id) → Activator.CreateInstance
handler.message = msg; handler.MsgHandle()

```

↓

```

业务分发层 (LoginManager)
LoginManager: event onLoginResult / onRegisterResult
Handler → RaiseXxx() | UI → 订阅事件

```

↓

```

UI 层 (LoginPanel)
订阅事件 → 显示结果 → result==0 → SceneManager.LoadScene

```

机制一：二进制协议格式 —— 4 字节 ID + 4 字节 Length + Body

每条消息的前 8 字节是固定头：[int32: 协议ID][int32: Body长度]，后面紧跟 Body 字节。序列化与反序列化分别由子类的 `Writing()` 和 `Reading()` 实现，字段按定义顺序依次写入/读取。

协议定义 (XML 驱动)

所有协议在 [ProtocolInfo.xml](#) 中统一定义，编辑器工具一键生成 C# 代码：

```

<!-- 登录请求 -->
<message id="1004" name="LoginMsg" namespace="GameLogin">
  <field type="string" name="account"/>
  <field type="string" name="password"/>
</message>

<!-- 登录响应: result=0 成功, 非0 失败, info=提示文字 -->
<message id="1006" name="LoginResultMsg" namespace="GameLogin">
  <field type="int" name="result"/>
  <field type="string" name="info"/>
</message>

```

序列化 (Writing) —— 以 LoginMsg 为例

```

// LoginMsg.cs — 由 ProtocolTool 从 XML 自动生成
public class LoginMsg : BaseMsg
{
    public string account;
    public string password;

    public override int GetBytesNum()
    {
        int num = 8; // 8 字节固定头
        num += 4 + Encoding.UTF8.GetByteCount(account); // string = 4字节长度前
    }
}

```

```
    缀 + UTF8字节
    num += 4 + Encoding.UTF8.GetByteCount(password);
    return num;
}

public override byte[] Writing()
{
    int index = 0;
    byte[] bytes = new byte[GetBytesNum()];
    WriteInt(bytes, GetID(), ref index);           // [0..3]  协议ID =
1004
    WriteInt(bytes, bytes.Length - 8, ref index);   // [4..7]  Body长度
    WriteString(bytes, account, ref index);         // [8..]   account
    字段
    WriteString(bytes, password, ref index);        // [...]   password
    字段
    return bytes;
}
}
```

反序列化（Reading）—— 对称读取

```
public override int Reading(byte[] bytes, int beginIndex = 0)
{
    int index = beginIndex;
    account = ReadString(bytes, ref index);
    password = ReadString(bytes, ref index);
    return index - beginIndex; // 返回读取了多少字节
}
```

BaseData 提供的序列化原语（全部用 BitConverter + Encoding.UTF8，小端序）：

写入	读取	字节数
WriteInt	ReadInt	4
WriteShort	ReadShort	2
WriteLong	ReadLong	8
WriteFloat	ReadFloat	4
WriteBool	ReadBool	1
WriteString	ReadString	4 + UTF8长度（先写长度再写内容）
WriteData<T>	ReadData<T>	嵌套 BaseData

机制二：分包/粘包处理

TCP 是流式协议，一次收到的字节可能是半条消息（分包），也可能是多条消息粘在一起（粘包）。解决方案：1MB 缓存区 + 循环解析头 + 完整性校验 + 未完成数据前移。

```
// NetAsyncMgr.HandleReceiveMsg() — 核心粘包处理
private byte[] cacheBytes = new byte[1024 * 1024]; // 1MB 累积缓存
private int cacheNum = 0; // 缓存中有效字节数

private void HandleReceiveMsg(int receiveNum)
{
    int msgID = 0, msgLength = 0, nowIndex = 0;
    cacheNum += receiveNum; // 累加新收到的字节

    while (true)
    {
        msgLength = -1;
        // ① 头部可读 (>= 8 字节) → 读 ID 和 Length
        if (cacheNum - nowIndex >= 8)
        {
            msgID = BitConverter.ToInt32(cacheBytes, nowIndex);
            nowIndex += 4;
            msgLength = BitConverter.ToInt32(cacheBytes, nowIndex);
            nowIndex += 4;
        }

        // ② Body 完整到达 → 解析消息并入队
        if (cacheNum - nowIndex >= msgLength && msgLength != -1)
        {
            BaseMsg baseMsg = msgPool.GetMessage(msgID); // 反射创建消息
            baseMsg.Reading(cacheBytes, nowIndex); // 反序列化 Body
            BaseHandler baseHandler = msgPool.GetHandler(msgID); // 反射创建

            Handler
            baseHandler.message = baseMsg;
            receiveQueue.Enqueue(baseHandler); // 入队等待主线程处

            nowIndex += msgLength;
            if (nowIndex == cacheNum) { cacheNum = 0; break; } // 刚好用完，退出
        }
        else
        {
            // ③ 消息不完整（半包）→ 剩余字节前移到缓存头部，等待下次拼接
            if (msgLength != -1) nowIndex -= 8; // 回退已读的头部
            Array.Copy(cacheBytes, nowIndex, cacheBytes, 0, cacheNum - nowIndex);
            cacheNum = cacheNum - nowIndex;
            break;
        }
    }
}
```

三种情况示意：

收到 30 字节: [ID LEN=10 BODY(10B)] [ID LEN=4 BODY(4B)]	← 粘包, 循环解析两次
收到 5 字节: [ID LEN=10 BO]	← 半包, 等下次
第二次收 8 字节: [DY(7B)]	← 拼接完成
收到 8 字节: [ID LEN=10]	← 头部刚好, 等 Body

机制三：线程模型 —— 子线程收包入队，主线程出队分发

Unity API 只能在主线程调用。 `SocketAsyncEventArgs.Completed` 回调在 .NET 线程池执行，所以子线程只做「解析 + 入队」，`Update()` 在主线程做「出队 + 调用 Handler」。

子线程 (ReceiveCallback)	主线程 (Update)
socket.ReceiveAsync 完成	
HandleReceiveMsg()	
解析头部 + 校验完整性	
反射创建 Msg + Handler	
receiveQueue.Enqueue(handler)	
socket.ReceiveAsync 再次发起	
	if (receiveQueue.Count > 0)
receiveQueue.Dequeue().MsgHandle()	
	↑ 在主线程序列化调用 Unity API

```
// 接收回调 — 运行在 .NET 线程池
private void ReceiveCallback(object obj, SocketAsyncEventArgs args)
{
    if (args.SocketError == SocketError.Success)
    {
        HandleReceiveMsg(args.BytesTransferred); // 解析 + 入队
        args.SetBuffer(cacheNum, args.Buffer.Length - cacheNum);
        if (this.socket != null && this.socket.Connected)
            socket.ReceiveAsync(args); // 继续异步接收
    }
}

// Update — 每帧在主线程执行
void Update()
{
    if (receiveQueue.Count > 0)
        receiveQueue.Dequeue().MsgHandle(); // Handler 逻辑在主线程, 安全调用 Unity
API
}
```

发送走 `SendAsync`，也是异步的。心跳通过 `InvokeRepeating("SendHeartMsg", 0, 2)` 每 2 秒在主线程触发。

机制四：协议 ID 路由 —— MsgPool + 反射

MsgPool 维护 `Dictionary<int, Type>` 的 ID→消息类型 和 ID→Handler 类型映射。收到消息时通过 **Activator.CreateInstance** 反射创建实例，完全消除 `switch-case`。

```
// MsgPool.cs
public class MsgPool
{
    private Dictionary<int, Type> messsages = new Dictionary<int, Type>();
    private Dictionary<int, Type> handlers = new Dictionary<int, Type>();

    public MsgPool()
    {
        Register(1001, typeof(RegisterMsg),      typeof(RegisterMsgHandler));
        Register(1004, typeof(LoginMsg),          typeof(LoginMsgHandler));
        Register(1005,
typeof(RegisterResultMsg),typeof(RegisterResultMsgHandler));
        Register(1006, typeof(LoginResultMsg),    typeof(LoginResultMsgHandler));
        Register(1002, typeof(HeartMsg),          typeof(HeartMsgHandler));
        Register(1003, typeof(QuitMsg),           typeof(QuitMsgHandler));
    }

    public BaseMsg GetMessage(int id) =>
        Activator.CreateInstance(messsages[id]) as BaseMsg;

    public BaseHandler GetHandler(int id) =>
        Activator.CreateInstance(handlers[id]) as BaseHandler;
}
```

对比被替换掉的方案（`HandleReceiveMsg` 中的注释掉的代码）：

```
// 旧方案：每次新增协议都要来这加 switch-case
switch (msgID) {
    case 1001: baseMsg = new PlayerMsg(); baseMsg.Reading(...); break;
    case 1004: baseMsg = new LoginMsg();  baseMsg.Reading(...); break;
    // ... 每加一个协议加两行
}
// 新方案：一行 Register，NetAsyncMgr 一行不改
BaseMsg baseMsg = msgPool.GetMessage(msgID);
```

代码生成器每次都重新生成 `MsgPool.cs`（覆盖），确保 `Register` 调用与 XML 始终同步。

机制五：LoginManager 事件解耦网络层与 UI

Handler 不知道 UI 存在，UI 不知道网络怎么收发包。中间通过 **LoginManager** 的 C# event 桥接。


```
// LoginManager.cs — 中介者 + 事件中心
public class LoginManager : BaseManager<LoginManager>
{
    public event Action<int, string> onLoginResult;
    public event Action<int, string> onRegisterResult;

    public void Login(string account, string password)
    {
        LoginMsg msg = new LoginMsg();
        msg.account = account; msg.password = password;
        NetAsyncMgr.Instance.Send(msg);
    }

    public void RaiseLoginResult(int result, string info)
        => onLoginResult?.Invoke(result, info);
}
```

```
// LoginResultMsgHandler.cs — Handler 只调 Raise, 不碰 UI
public class LoginResultMsgHandler : BaseHandler
{
    public override void MsgHandle()
    {
        LoginResultMsg msg = message as LoginResultMsg;
        LoginManager.GetInstance().RaiseLoginResult(msg.result, msg.info);
    }
}
```

```
// LoginPanel.cs — UI 只订阅事件, 不碰网络
void Start()
{
    LoginManager.GetInstance().onLoginResult += OnLoginResult;
    LoginManager.GetInstance().Connect(serverIp, serverPort);
}

void OnLoginResult(int result, string info)
{
    SetTip(info);
    if (result == 0)
        SceneManager.LoadScene(firstSceneName); // 登录成功 → 跳场景
}
```

数据流完整链路（登录为例）：

```
LoginPanel.OnLoginClick()
    → LoginManager.Login(account, password)
    → new LoginMsg + NetAsyncMgr.Send(msg)
```

```

→ 服务器处理 → 返回 LoginResultMsg(ID=1006)
→ ReceiveCallback(子线程) → HandleReceiveMsg → 读头部 ID=1006
→ msgPool.GetMessage(1006) → new LoginResultMsg → Reading()
→ msgPool.GetHandler(1006) → new LoginResultMsgHandler
→ receiveQueue.Enqueue(handler)
→ Update()(主线程) → handler.MsgHandle()
→ LoginManager.RaiseLoginResult(result, info)
→ LoginPanel.OnLoginResult → result==0 → SceneManager.LoadScene("Scene01")

```

机制六：心跳保活 + DontDestroyOnLoad

```

// NetAsyncMgr.Awake()
void Awake()
{
    instance = this;
    DontDestroyOnLoad(this.gameObject); // 跨场景不销毁
    InvokeRepeating("SendHeartMsg", 0, SEND_HEART_MSG_TIME); // 每2秒发心跳
}

private void SendHeartMsg()
{
    if (socket != null && socket.Connected)
        Send(heartMsg); // HeartMsg: ID=1002, body 为空
}

```

- **DontDestroyOnLoad**：登录场景的 **NetAsyncMgr** GameObject 在 **SceneManager.LoadScene("Scene01")** 后仍存活，Socket 连接不中断
- **HeartMsg** (ID=1002)：body 为空，只占 8 字节，每 2 秒发送一次
- 优雅关闭：**OnDestroy()** 时调用 **Close(true)**，先发 **QuitMsg** (ID=1003) 通知服务器，再 **Shutdown** → **Disconnect** → **Close**

代码生成流程



Handler 的「生成一次，后续手写」策略：代码生成器首次生成 Handler 骨架（只有 `MsgHandle()` 空方法），后续 XML 变更重新生成时跳过已有 Handler 文件，开发者手写的业务逻辑（如 `LoginResultMsgHandler` 中调用 `RaiseLoginResult`）不会被覆盖。

关键文件速查

文件	角色
<code>Net/NetAsyncMgr.cs</code>	核心网络引擎：连接、收发、粘包处理、心跳、线程调度
<code>Net/Base/BaseData.cs</code>	序列化基类：WriteInt/ReadString 等原语
<code>Net/Base/BaseMsg.cs</code>	消息基类： GetBytesNum/Writing/Reading/GetID
<code>Net/Base/BaseHandler.cs</code>	Handler 基类：message 字段 + MsgHandle() 抽象方法
<code>Net/LoginManager.cs</code>	登录业务中介：事件 + Connect/Login/Register
<code>Net/Base/BaseManager.cs</code>	泛型懒汉单例基类
<code>Protocol/Pool/MsgPool.cs</code>	ID→Type 映射字典 + Activator.CreateInstance 反射路由
<code>Protocol/GameLogin/Msg/LoginMsg.cs</code>	登录请求消息（代码生成）
<code>Protocol/GameLogin/Msg/LoginResultMsg.cs</code>	登录响应消息（代码生成）
<code>Protocol/GameLogin/Msg/LoginResultMsgHandler.cs</code>	登录响应处理（手写业务逻辑）
<code>Protocol/GameSystem/Msg/HeartMsg.cs</code>	心跳消息（body 为空）
<code>UI/LoginPanel.cs</code>	登录 UI：输入、按钮、订阅事件、场景跳转
<code>Editor/ProtocolTool/ProtocolInfo.xml</code>	协议唯一定义源
<code>Editor/ProtocolTool/GenerateCSharp.cs</code>	代码生成器（XML → C#）